Python 8-Puzzle Solver- Search Algorithms & Heuristics

Project Based Learning (PBL) Report
for the course
Artificial Intelligence -20CS31003

BACHELOR OF TECHNOLOGY
IN

COMPUTER SCIENCE AND ENGINEERING

By

Thanniru Dharma Nithin 21R11A6692

Under the guidance of

Mr. Shaik Akbar

Associate Professor, CSE(AI&ML) Department

Department of Computer Science and Engineering
Accredited by NBA

Geethanjali College of Engineering and Technology
(UGC Autonomous)
(Affiliated to J.N.T.U.H, Approved by AICTE, New Delhi)
Cheeryal (V), Keesara (M), Medchal.Dist.-501 301.

January-2024

Geethanjali College of Engineering and Technology

Department of Computer Science and Engineering (AI&ML)

(UGC AUTONOMOUS INSTITUTION)

Accredited by NAAC with 'A+' Grade & NBA, Approved by AICTE and Affiliated to JNTUH

Cheeryal (V), Keesara (M), Medchal (Dist), Telangana – 501 301.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI&ML)

CERTIFICATE

This is to certify that the Artificial Intelligence (PBL) Report entitled **"Python 8-Puzzle Solver- Search Algorithms & Heuristics "** is a bonafide work done and submitted by **Thanniru Dharma Nithin(21R11A6692)** during the academic year 2023 – 2024, in partial fulfilment of requirement forthe award of Bachelor of Technology degree in **"Computer Science and Engineering (AI&ML)"** from Geethanjali College of Engineering and Technology (Accredited by NAAC with 'A+' Grade & NBA, Approved by AICTE and Affiliated to JNTUH), is a bonafide recordof work carried out by them under guidance and supervision.

Certified further that to my best of the knowledge, the work in this project has not been submitted to any other institution for the award of any degree or diploma.

FACULTY GUIDE	HEAD OF THE DEPARTMENT
Mr. Shaik Akbar	**Dr. L. Venkateswarlu**
Associate Professor **CSE(AI&ML) Department**	**Professor & HOD** **CSE(AI&ML) Department**

DECLARATION

We hereby declare that the project report entitled **"Python 8-Puzzle Solver- Search Algorithms & Heuristics"** is an original work done and submitted to CSE(AI&ML) Department, from Geethanjali College of Engineering and Technology (Accredited by NAAC with 'A+' Grade & NBA, Approved by AICTE and Affiliated to JNTUH), in partial fulfilment of the requirement for the award of Bachelor of Technology in **"Computer Science and Engineering (AI&ML)"** and it is a record of bonafide project work carried out by us under the guidance of **Mr. Shaik Akbar, Associate Professor, Department of CSE(AI&ML)**.

Signature of the Student

Thanniru Dharma Nithin

(21R11A6692)

ACKNOWLEDGEMENT

This satisfaction of completing this project would be incomplete without mentioning our gratitude towards all the people who have supported us. Constant guidance and encouragement have been instrumental in the completion of this project.

First and foremost, we thank the chairman, Principal ,Vice Principal for availing infrastructural facilities to complete the project in time.

We offer our sincere gratitude to our faculty guide Mr. Shaik Akbar, Associate Professor, CSE(AI&ML) Department, from Geethanjali College of Engineering and Technology (Accredited by NAAC with 'A+' Grade & NBA, Approved by AICTE and Affiliated to JNTUH), for his immense support, timely co-operation and valuable advice throughout the course of our project work.

We would like to thank the Head of Department, Dr. L. Venkateswarlu, for his meticulous care and co-operation throughout the project work.

We are thankful to all the project Coordinators for their supportive guidance and for having provided the necessary help for carrying forward this project without any obstacle and hindrances.

ABSTRACT

The 8-puzzle, a classic problem in the realm of artificial intelligence and computational algorithms, serves as a cornerstone for understanding various search strategies. This report presents an in-depth exploration and implementation of three prominent search algorithms- Breadth-First Search (BFS), Depth-First Search (DFS), and the A* algorithm, specifically designed for solving the 8-puzzle problem. This Python script offers a solution to the intriguing 8-puzzle challenge, which features a 3x3 grid containing 8 numbered tiles and an empty space. The primary objective is to strategically slide these tiles, utilizing three distinct search methods: Breadth-First Search (BFS), Depth-First Search (DFS), and the sophisticated A* Search algorithm. Within A* Search, the approach leverages two heuristic techniques- Manhattan Distance and Euclidean Distance, which provide intelligent ways to determine optimal moves. Users have the flexibility to specify an initial puzzle configuration and select their preferred solving algorithm. The solver employs various data structures, including queues, sets, and priority queues. Additionally, the script calculates metrics such as the number of nodes explored, maximum depth reached, and execution time for each algorithm. The puzzle's solvability is checked using inversion counting. The output includes the solution path, the number of nodes explored, maximum depth, and execution time. The provided search algorithms and heuristics offer a comprehensive exploration of different strategies for solving the 8-puzzle problem.

Keywords: Breadth-First Search (BFS), Depth-First Search (DFS), A* algorithm, Manhattan Distance and Euclidean Distance

TABLE OF CONTENTS

S.No.	Contents	Page No
1	Introduction	1
2	System Analysis	3
3	System Design	6
4	Implementation	9
5	Output	14
6	Conclusion and Future Scope	19
7	Bibliography	20

1.Introduction

1.1 Defining the Game

The 8-puzzle, a renowned sliding puzzle game, is played on a 3x3 grid containing numbered tiles from 1 to 8 and an empty space. The player's objective is to rearrange the tiles by sliding them into the empty space, aiming to reach a predetermined goal configuration. This seemingly simple yet challenging game has captivated the interest of researchers and enthusiasts alike due to its applicability in algorithmic problem-solving.

1.2 Motivation

The motivation behind this project lies in the educational and practical significance of the 8-puzzle. By developing a solver that employs various search algorithms and heuristics, we aim to offer a hands-on experience in algorithmic problem-solving. This project serves as a practical application of theoretical concepts, making the intricacies of search algorithms tangible and engaging.

1.3 Objectives

Implementing Algorithms-

Develop implementations of Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search algorithms tailored for solving the 8-puzzle.

Heuristic Integration-

Integrate heuristic functions, specifically Manhattan Distance and Euclidean Distance, into the A* Search algorithm. These heuristics will guide the search process, enhancing efficiency.

User-Friendly Interface-

Create an intuitive and user-friendly interface allowing users to input initial puzzle states and select algorithms seamlessly. The interface enhances accessibility for both novice and experienced users.

Metrics and Solution Path-

Implement a comprehensive metrics system that calculates and displays key performance indicators, such as nodes explored and execution time. Additionally, present the solution path, offering insights into the sequence of moves leading to the goal state.

Memory Usage Monitoring-

Employ the psutil library to monitor memory usage during the execution of algorithms. Understanding the memory requirements contributes to a holistic evaluate on of algorithmic efficiency.

2. System Analysis

The system analysis phase involves a comprehensive examination of the requirements and constraints that influence the design and development of the 8-puzzle solver. This section outlines the functional specifications, performance requirements, and the necessary software and hardware components essential for the successful implementation of the project.

2.1 Functional Requirement Specification

The functional requirements of the 8-puzzle solver are defined by the key capabilities it must possess to meet its objectives. These requirements include:

Algorithm Implementations-

Implement Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search algorithms for solving the 8-puzzle. Each algorithm must efficiently explore the solution space and find an optimal solution.

Heuristic Integration-

Integrate two heuristic functions, namely Manhattan Distance and Euclidean Distance, into the A* Search algorithm. These heuristics should guide the search process to improve efficiency.

User Interface-

Develop a user-friendly interface allowing users to input the initial puzzle state, choose the desired algorithm, and interact with the solver effortlessly.

Metrics Calculation-

Implement a metrics system to calculate and display key performance indicators such as the number of nodes explored, maximum depth reached, and execution time for each algorithm.

Solution Path Presentation-

Design a feature to present the solution path in a readable format, allowing users to understand the sequence of moves leading to the goal state.

Memory Usage Monitoring-

Utilize the psutil library to monitor memory usage during algorithm execution, providing insights into the memory requirements of each algorithm.

2.2 Performance Requirements

The performance requirements define the criteria that the 8-puzzle solver must meet to ensure efficient and effective functionality. These include

Execution Time-

The solver should provide solutions within a reasonable timeframe, ensuring timely responses to user inputs.

Memory Usage-

Algorithms should be optimized to use memory efficiently, and the solver should not impose excessive memory requirements on the user's system.

Solution Optimality-

Algorithms, particularly A* Search, should strive to find optimal solutions with the fewest moves possible.

User Interface Responsiveness:

The user interface should be responsive, providing real-time updates and feedback during algorithm execution.

2.3 Software Requirements

The software requirements detail the tools, libraries, and frameworks essential for the development and execution of the 8-puzzle solver. These include-

Programming Language-

Implement the solver using a programming language such as Python, chosen for its readability, versatility, and the availability of relevant libraries.

Graphical User Interface (GUI) Library-

Utilize a GUI library (e.g., Tkinter) for designing an interactive and user-friendly interface.

psutil Library-

Integrate the psutil library to monitor memory usage during algorithm execution.

2.4 Hardware Requirements

The hardware requirements outline the minimal specifications necessary for running the 8-puzzle solver. These include-

Processor-

A processor capable of handling computational tasks involved in algorithm execution.

Memory (RAM)-

Adequate RAM to accommodate the memory requirements of the solver.

3. System Design

The system design phase focuses on architecting the 8-puzzle solver, delineating its architecture and modular structure. This section outlines the architectural design and key modules that collectively form the foundation of the entire system.

3.1 Architecture Design

The architecture of the 8-puzzle solver is designed to accommodate various search algorithms, heuristic functions, and user interactions. The system follows a modular and flexible architecture to facilitate easy integration of new algorithms or features. The primary architectural components include-

User Interface (UI) Module-

Responsible for interacting with users, capturing initial puzzle states, and algorithm/heuristic selections. Facilitates a seamless and intuitive user experience.

Algorithmic Solver Module-

Encompasses the implementation of Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search algorithms.
Enables the solver to navigate through the puzzle space efficiently.

Heuristic Module-

Houses the Manhattan Distance and Euclidean Distance heuristic functions.
Provides essential estimations for guiding the A* Search algorithm.

Metrics and Solution Path Module-

Handles the calculation and presentation of key performance metrics, including nodes explored, maximum depth, and execution time.Manages the solution path for display and analysis.

Memory Monitoring Module-

Utilizes the psutil library to monitor memory usage during algorithm execution.

Enhances the understanding of the system's memory requirements.

3.2 Modules

The system is structured into modular components, each serving a distinct purpose and contributing to the overall functionality of the 8-puzzle solver.

User Interface (UI) Module-

Responsibilities-

Capture and validate user input for the initial puzzle state.

Provide a menu for users to select the desired algorithm and heuristic (if applicable).

Facilitate a user-friendly interaction flow.

Algorithmic Solver Module-

Responsibilities-

Implement Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search algorithms.

Manage the systematic exploration of the puzzle space.

Interface with the heuristic module in the case of A* Search.

Heuristic Module-

Responsibilities-

House the Manhattan Distance and Euclidean Distance heuristic functions.

Calculate heuristic values for puzzle states, contributing to the A* Search algorithm's efficiency.

Metrics and Solution Path Module-

Responsibilities-

Calculate and store metrics such as nodes explored, maximum depth, and execution time.

Manage the solution path for presentation.

Memory Monitoring Module-

Responsibilities-

Utilize the psutil library to monitor and record memory usage during algorithm execution.

Provide insights into the memory requirements of the system.

Integration Points-

- The modular design allows for seamless integration and communication between different components-
- The UI module interacts with the Algorithmic Solver, allowing users to choose algorithms and providing initial puzzle states.
- The Algorithmic Solver interfaces with the Heuristic Module when employing A* Search, utilizing heuristic values for informed decision-making
- The Metrics and Solution Path Module collects data from the Algorithmic Solver, presenting key performance metrics and the solution path.
- This modular architecture ensures maintainability, scalability, and extensibility, making it straightforward to incorporate new algorithms or features into the solver.

Potential Extensions-

Future iterations of the system could consider the following extensions-

Advanced Algorithm Modules-

Integration of more sophisticated search algorithms or algorithmic variants.

Enhanced Heuristic Functions-

Introduction of additional heuristic functions to explore their impact on algorithmic efficiency.

Graphical User Interface (GUI)-

Transition from a console-based UI to a graphical interface for enhanced user experience.

The modular design lays the groundwork for accommodating these extensions seamlessly.

4. Implementation

In this section, we present the individual implementations of the search algorithms used in the 8-puzzle solver: Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search.

4.1 Breadth-First Search (BFS) Algorithm

```
from collections import deque

def bfs(initial_state):

    frontier = deque([initial_state])

    explore = set()

    parent_map = {}

    while frontier:

        state = frontier.popleft()

        explore.add(state

        if state == "012345678":

            return find_path(parent_map, initial_state)

        else:

            for child in generate_children(state):

                if child not in explore:

                    parent_map[child] = state

                    frontier.append(child)

    return []

# Additional functions for BFS

def find_path(parent_map, initial_state):

    current_state = "012345678"

    path = [current_state

    while current_state != initial_state:

        path.append(parent_map[current_state])
```

```
        current_state = parent_map.get(current_state
    return path[::-1
# Other functions (e.g., generate_children) remain unchanged
```

4.2 Depth-First Search (DFS) Algorithm

```
from collections import deque
def dfs(initial_state):
    frontier = deque([initial_state])
    explore = set()
    parent_map = {}
    while frontier:
        state = frontier.pop()
        explore.add(state)
        if state == "012345678":
            return find_path(parent_map, initial_state)
        else:
            for child in reversed(generate_children(state)):
                if child not in explore:
                    parent_map[child] = state
                    frontier.append(child
    return []
# Additional functions for DFS
def find_path(parent_map, initial_state):
    current_state = "012345678"
    path = [current_state]
    while current_state != initial_state:
        path.append(parent_map[current_state])
        current_state = parent_map.get(current_state)
```

```
    return path[::-1]
```

4.3 A* Search Algorithm

python

Copy code

```
import queue


def A_star(initial_state, heuristic):
    frontier = queue.PriorityQueue()
    frontier.put([heuristic(initial_state), initial_state])
    explore = set()
    parent_map = {}
    while not frontier.empty():
        state1 = frontier.get()
        prev_cost = state1[0]
        state = state1[1]
        explore.add(state)
        if state == "012345678":
            return find_path(parent_map, initial_state)
        for child in generate_children(state):
            if child not in frontier.queue and child not in explore:
                parent_map[child] = state
                frontier.put([heuristic(child) + prev_cost + 1, child])
    return []
# Additional functions for A*
def find_path(parent_map, initial_state):
    current_state = "012345678"
    path = [current_state]
    while current_state != initial_state:
```

```
        path.append(parent_map[current_state])
        current_state = parent_map.get(current_state)
    return path[::-1]
```

4.4 User Guided Interface

```
def solver(initial_state, algorithm, heuristic=None):
    solution = None
    if algorithm == 1:
        solution = bfs(initial_state)
    elif algorithm == 2:
        solution = dfs(initial_state)
    elif algorithm == 3:
        if heuristic == 1:
            solution = A_star(initial_state, manhattan_distance)
        elif heuristic == 2:
            solution = A_star(initial_state, euclides_distance)
    end_mem = psutil.virtual_memory().used
    return solution, end_mem
if _name_ == '_main_':
    print("Welcome to 8_puzzle solver")
    while True:
        while True:
            initial_state = input("Insert initial state: ")
            freq = Counter(initial_state)
            if len(initial_state) == len(freq) == 9 and initial_state.isdecimal():
                break
            print("Wrong initial state")
```

```
print("Algorithms\n"
      "-----------\n"
      "[1] BFS\n"
      "[2] DFS\n"
      "[3] A*\n")
while True:
    switch = int(input("=>"))
    if switch in range(1, 4):
        break
    print("Invalid choice"
if not is_solvable(initial_state):
    print(initial_state + " Cannot be solved")
    continue
if switch == 1 or switch == 2:
    solution = solver(initial_state, switch)
elif switch == 3:
    while True:
        h = int(input("Choose heuristic:\n"
                      "-----------------\n"
                      "[1] Manhattan\n"
                      "[2] Euclidean\n"))
        if h in [1, 2]:
            break
        print("Invalid choice")
    if h in [1, 2]:
        solution = solver(initial_state, switch, h)
```

These algorithmic implementations are isolated for clarity and can be utilized individually based on specific requirements. Each algorithm contributes to the versatility of the 8-puzzle solver, offering different approaches to solving the problem.

5.Output

- **BFS interface**

```
Welcome to 8_puzzle solver
Insert initial state: 123405678
Algorithms
-----------
[1] BFS
[2] DFS
[3] A*

=>1
1 2 3
4 0 5
6 7 8
---------------------------------------------
1 2 3
0 4 5
6 7 8
---------------------------------------------
0 2 3
1 4 5
6 7 8
---------------------------------------------
2 0 3
1 4 5
6 7 8
---------------------------------------------
2 3 0
1 4 5
6 7 8
---------------------------------------------
2 3 5
1 4 0
6 7 8
---------------------------------------------
2 3 5
1 0 4
6 7 8
---------------------------------------------
2 0 5
1 3 4
6 7 8
---------------------------------------------
0 2 5
1 3 4
6 7 8
```

Fig- 5.1 BFS Input

```
2 3 0
1 4 5
6 7 8
---------------------------------------------
2 3 5
1 4 0
6 7 8
---------------------------------------------
2 3 5
1 0 4
6 7 8
---------------------------------------------
2 0 5
1 3 4
6 7 8
---------------------------------------------
0 2 5
1 3 4
6 7 8
---------------------------------------------
1 2 5
0 3 4
6 7 8
---------------------------------------------
1 2 5
3 0 4
6 7 8
---------------------------------------------
1 2 5
3 4 0
6 7 8
---------------------------------------------
1 2 0
3 4 5
6 7 8
---------------------------------------------
1 0 2
3 4 5
6 7 8
---------------------------------------------
0 1 2
3 4 5
6 7 8
---------------------------------------------
Cost = 14
Depth = 15
Nodes Explored = 5162
Running Time = 0.07274985313415527
Insert initial state:
```

Fig -5.2 BFS Output

- **DFS Interface**

```
Insert initial state: 123405678
Algorithms
-----------
[1] BFS
[2] DFS
[3] A*

=>2
```

Fig-5.3 DFS Input

```
3 2 5
0 1 8
6 7 4
---------------------------------------------
3 2 5
6 1 8
0 7 4
---------------------------------------------
3 2 5
6 1 8
7 0 4
---------------------------------------------
3 2 5
6 1 8
7 4 0
---------------------------------------------
3 2 5
6 1 0
7 4 8
---------------------------------------------
3 2 0
6 1 5
7 4 8
---------------------------------------------
3 0 2
6 1 5
7 4 8
---------------------------------------------
3 1 2
6 0 5
7 4 8
---------------------------------------------
3 1 2
6 4 5
7 0 8
---------------------------------------------
3 1 2
6 4 5
0 7 8
---------------------------------------------
3 1 2
0 4 5
6 7 8
---------------------------------------------
0 1 2
3 4 5
6 7 8
---------------------------------------------
Cost = 34838
Depth = 34838
Nodes Explored = 37067
Running Time = 0.35698962211608887
Insert initial state:
```

Fig – 5.4 DFS Output

- **A* Using Heuristic Distance search Interface**

  - **Manhattan Process**

```
Welcome to 8_puzzle solver
Insert initial state: 123405678
Algorithms
-----------
[1] BFS
[2] DFS
[3] A*

=>3
Choose heuristic:
----------------
[1] Manhattan
[2] Euclidean
1
1 2 3
4 0 5
6 7 8
--------------------------------------------
1 2 3
0 4 5
6 7 8
--------------------------------------------
0 2 3
1 4 5
6 7 8
--------------------------------------------
2 0 3
1 4 5
6 7 8
--------------------------------------------
2 3 0
1 4 5
6 7 8
--------------------------------------------
2 3 5
1 4 0
6 7 8
--------------------------------------------
2 3 5
1 0 4
6 7 8
--------------------------------------------
2 0 5
1 3 4
6 7 8
```

Fig-5.5 Manhattan Input

```
1 4 5
6 7 8
--------------------------------------------
2 3 0
1 4 5
6 7 8
--------------------------------------------
2 3 5
1 4 0
6 7 8
--------------------------------------------
2 3 5
1 0 4
6 7 8
--------------------------------------------
2 0 5
1 3 4
6 7 8
--------------------------------------------
0 2 5
1 3 4
6 7 8
--------------------------------------------
1 2 5
0 3 4
6 7 8
--------------------------------------------
1 2 5
3 0 4
6 7 8
--------------------------------------------
1 2 5
3 4 0
6 7 8
--------------------------------------------
1 2 0
3 4 5
6 7 8
--------------------------------------------
1 0 2
3 4 5
6 7 8
--------------------------------------------
0 1 2
3 4 5
6 7 8
--------------------------------------------
Cost = 14
Depth = 14
Nodes Explored = 483
Running Time = 0.029794692993164062
Insert initial state: 
```

Fig – 5.6 Manhattan Output

- **Euclidean Process**

```
Welcome to 8_puzzle solver
Insert initial state: 123405678
Algorithms
-----------
[1] BFS
[2] DFS
[3] A*

=>3
Choose heuristic:
-----------------
[1] Manhattan
[2] Euclidean
2
1 2 3
4 0 5
6 7 8
--------------------------------------------
1 2 3
0 4 5
6 7 8
--------------------------------------------
0 2 3
1 4 5
6 7 8
--------------------------------------------
2 0 3
1 4 5
6 7 8
--------------------------------------------
2 3 0
1 4 5
6 7 8
--------------------------------------------
2 3 5
1 4 0
6 7 8
--------------------------------------------
2 3 5
1 0 4
6 7 8
--------------------------------------------
2 0 5
1 3 4
6 7 8
--------------------------------------------
```

Fig -5.7 Euclidean Input

```
--------------------------------------------
1 2 5
0 3 4
6 7 8
--------------------------------------------
1 2 5
3 0 4
6 7 8
--------------------------------------------
1 2 5
3 4 0
6 7 8
--------------------------------------------
1 2 0
3 4 5
6 7 8
--------------------------------------------
1 0 2
3 4 5
6 7 8
--------------------------------------------
0 1 2
3 4 5
6 7 8
--------------------------------------------
Cost = 14
Depth = 14
Nodes Explored = 540
Running Time = 0.028583049774169922
Insert initial state:
```

Fig -5.8 Euclidean Input

6. Conclusion and Future Scope

6.1 Conclusion-

The 8-puzzle solver project has demonstrated the successful implementation of various search algorithms and heuristic functions to address the classic 8-puzzle problem. Through the exploration of Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search algorithms, the solver provides valuable insights into their efficiency and applicability.

Key accomplishments include-

- **Algorithmic Solutions-**The solver can successfully navigate through the puzzle space and find optimal solutions using different algorithms.
- **Heuristic Integration-** The inclusion of heuristic functions, such as Manhattan Distance and Euclidean Distance, enhances the A* Search algorithm's performance by guiding the search toward more promising paths.
- **Modular and Extensible Design-** The system's modular architecture allows for easy integration of new algorithms, heuristics, and potential enhancements.

6.2 Future Scope-

As we conclude this project, several avenues for future exploration and improvement emerge:

- Advanced Algorithms- Consider integrating more advanced search algorithms or algorithmic variants to further enhance the solver's capabilities. Exploring algorithms like IDA* or other informed search strategies could contribute to the project's depth.
- **Enhanced Heuristic Functions-** Investigate additional heuristic functions beyond Manhattan Distance and Euclidean Distance. Evaluating the impact of different heuristics on the A* Search algorithm's performance can lead to a more comprehensive understanding.
- **Graphical User Interface (GUI)-**Transition from a console-based user interface to a graphical interface to improve user experience. A visual representation of the puzzle and its solution path can enhance the solver's accessibility
- **Parallel Processing-**Explore parallel processing techniques to optimize the solver's execution time, especially for larger puzzle instances. Parallelizing certain aspects of the algorithms can lead to performance improvements
- **User Interaction Features-**Introduce features that allow users to interact with the solver during its execution. Real-time updates, progress indicators, and pause/resume functionalities can enhance user engagement.

In conclusion, the 8-puzzle solver project lays the groundwork for continuous improvement and exploration within the realm of puzzle-solving algorithms. Future endeavors can build upon this foundation to address more complex puzzles, incorporate cutting-edge techniques, and provide an even more user-friendly experience.

7.Bibliography

References for 8-Puzzle Problem and Search Algorithms-

- Russell, S., & Norvig, P. (2009). "Artificial Intelligence: A Modern Approach." Prentice Hall.
- Python For Everyone (Exploring Data in Python 3)